

Image Generative Models

XIANG Bojie

*Hong Kong Baptist University
25400444@life.hkbu.edu.hk*

ZHOU Tianchi

*Hong Kong Baptist University
25409638@life.hkbu.edu.hk*

ZHU Tianxiao

*Hong Kong Baptist University
25407732@life.hkbu.edu.hk*

SU Zinao

*Hong Kong Baptist University
25434675@life.hkbu.edu.hk*

ZHAO Zihao

*Hong Kong Baptist University
25418823@life.hkbu.edu.hk*

I. Introduction

Modern image generative models learn a data distribution and sample new images from it, typically by mapping latent variables to the image space. For this project, We made two conditional models based on **VAE** [1], **GAN** [2] respectively to synthesize handwritten characters, turning digits and letters into realistic handwritten-style images; Additionally we made a relative larger model based **DDPM**[3] to generate class-conditional natural images.

This report presents three image generative models in terms of model architecture, training process, and the analysis of their output quality based on FID score.

Dataset and preprocessing: **1) Handwritten dataset** from Hugging Face <https://huggingface.co/datasets/PranomVignesh/handwritten-characters-complete/tree/main>. For this dataset, we performed preprocessing by selecting 74,000 + images from the training dataset, splitting them into new training and validation sets with a ratio of 9:1, and saving them as numpy files. **2) Cifar-10** [4] was used in its original form.

Third-party libraries: **PyTorch**, **NumPy**, **Matplotlib**, and **pytorch_fid** <https://github.com/mseitzer/pytorch-fid>. Implementations of key algorithms referenced the following two books: 1) **Deep Learning from Scratch 5** <https://github.com/oreilly-japan/deep-learning-from-scratch-5>, and 2) **Machine Learning with PyTorch and Scikit-Learn** [5].

Our source code is available on GitHub: https://github.com/cqxbj/image_generative_model

II. Contribution

VAE: ZHOU Tianchi, ZHU Tianxiao

GAN: ZHAO Zihao, SU Zinao, XIANG Bojie

DDPM: XIANG Bojie

Other Functions: All members

III. VAE and GAN

In this section, we trained two models based on the VAE[1] and GAN[2] frameworks, respectively. Both models were trained on the preprocessed handwritten digit dataset. To ensure a relatively fair comparison of their output quality, we designed the decoder of the VAE and the generator of the GAN to have similar architectures.

A. VAE architecture and Training

Autoencoder is a neural network designed to learn an identity function in an unsupervised way to reconstruct the original input while compressing the data in the process so as to discover a more efficient and compressed representation. Its loss function consists of two parts: Reconstruction Loss and KL Divergence Loss.

For the model design, we employ a convolutional encoder-decoder architecture with a latent Gaussian distribution. The encoder consists of three convolutional blocks with channels [1, 16, 32, 64], using stride-2 convolutions for downsampling (reducing spatial dimensions from 32×32 to 16×16, then 8×8) and includes batch normalization and ReLU activation. The resulting 4096-dimensional feature vector is mapped to the parameters of the latent distribution

(mean and log variance) via two linear layers, yielding a 256-dimensional latent space. The decoder processes a concatenated vector of the latent sample(reparameterization is used while training) and class label embedding through a linear layer, reshaping the output to $64 \times 8 \times 8$ feature maps. Two transposed convolutional blocks (channels [64, 32, 16]) with stride-2 upsampling restore spatial dimensions to 32×32 , each followed by batch normalization and ReLU. A final 1×1 convolution with sigmoid activation produces the output image.

The VAE model was trained on the handwritten digit dataset for 500 epochs with a batch size of 128 and a KL loss weight of 0.3. The training and validation losses demonstrated a consistent decline throughout the training process, ending at approximately 24.0 and 24.4 respectively). No overfitting was observed. Furthermore, the model’s FID scores on the training and validation sets were notably close at **49.64** and **50.60**, respectively.

B. GAN architecture and Training

A GAN consists of two neural networks trained simultaneously in a competitive setting: 1) The Generator (G) learns to map a noise vector z to a synthetic data sample $G(z)$ intended to resemble the true data distribution. 2) The Discriminator (D) receives both real and generated samples and learns to distinguish between the two by outputting a probability indicating whether a given sample comes from the real dataset.

For the model design, we adopt an architecture composed of a generator–discriminator pair trained in an adversarial manner. The generator follows a convolutional design that transforms a 256-dimensional Gaussian noise vector into a 32×32 grayscale image conditioned on class labels. Class labels are embedded into 12-dimensional vectors and concatenated with the noise input before being projected through a fully connected layer into a spatial tensor of shape $64 \times 8 \times 8$. This latent feature map is then upsampled using two ConvTranspose2d layers with output channel sizes of [32, 16], doubling the spatial resolution at each stage—from 8×8 to 16×16 , and finally to 32×32 . A final 3×3 convolution with sigmoid activation produces the single-channel output image. Batch normalization is applied after each transposed convolution to stabilize training dynamics. The discriminator mirrors this structure in reverse. Input images are processed through two convolutional downsampling blocks with channels [16, 32], reducing the spatial resolution from 32×32 to 16×16 , and then to 8×8 . The resulting feature map is flattened into a 2048-dimensional vector and passed through a fully connected layer with 256 units. Label embeddings (12-dimensional) are concatenated with these image features, enabling the discriminator to perform class-conditional binary classification. The final classification head consists of two linear layers that output a scalar probability indicating whether the sample is real or generated.

The GAN model was trained for 50 epochs with a custom adversarial strategy to balance training, as the Discriminator was stronger than the Generator in our project. The strategy involved increasing the Generator’s learning rounds by one per epoch whenever its loss became excessively high, with the number of rounds constrained to the interval [3, 6]. Training records indicate that this strategy was effective for the first 10 epochs, but the gap between the Generator and Discriminator increased in the subsequent 40 epochs. In terms of outputs quality, the Generator’s FID scores on the training and validation sets were close, at around **27.31** and **28.34**, respectively.

C. Evaluation and Comparison

Based on the training results, the VAE demonstrates superior training stability but tends to produce blurry images. In contrast, the training process for GANs is inherently unstable. To achieve a relatively acceptable result, we experimented with multiple initial parameter sets. Despite these efforts, we still encountered mode collapse in some instances. For instance the generator failed to produce diverse outputs for certain classes of data.



Fig. 1 Training loss curves for VAE and GAN models



(a) Blurry VAE



(b) Mode Collapse GAN

Fig. 2 Weakness of VAE and GAN

```

GENERATED BY OUR VAE MODEL
1234567890
1234567890
ABCDEFGHIJKLMN0PQRS74VWXYZ
0BC/EF94IKLMNOPRST56UVWXYZ

COMP7015 GROUP 42 IMAGE GENERATIVE MODEL
1 ZHAO ZHAO 25418823
2 BOJIE XIANG 25400444
3 ZHAO SU 25434675
4 TIANCHU ZHOU 25409638
5 TIANXIAO ZHU 25407732

```

(a) VAE

```

GENERATED BY OUR GANs MODEL
1234567890
1234567890
ABCDEFGHIJKLMN0PQRS74VWXYZ
ABCDEFGHIJKLMN0PQRS74VWXYZ

COMP7015 GROUP 42 IMAGE GENERATIVE MODEL
1 ZHAO ZHAO 25418823
2 BOJIE XIANG 25400444
3 ZHAO SU 25434675
4 TIANCHU ZHOU 25409638
5 TIANXIAO ZHU 25407732

```

(b) GAN

Fig. 3 Outputs of VAE and GAN

IV. DDPM

In this section, we will implement a larger model based on the well-known DDPM[3], trained on the CIFAR-10 dataset. Similarly to the previous two models, this one is also designed as a conditional model.

The reason for choosing DDPM in this project is that the experimental results from our VAE and GAN models demonstrate that the VAE often produces blurry images and fails to achieve high-quality generation. Although the GAN can generate good quality images, its training process is often unstable, requiring repeated adjustments of the training parameters to balance the generator and the discriminator. In addition, GANs frequently suffer from mode collapse. Given these challenges, we have decided to adopt DDPM in this section to train a model capable of generating CIFAR images.

Based on our understanding, the DDPM model can be decomposed into two key components: 1) a diffusion process with hyperparameters beta (can also be learnable by utilizing reparameterization), which is used to add noise and later remove noise from the input images according to a given timestep T (ranging from 1 to T). 2) a trained model (typically U-Net) predicts the specific noise added at timestep T based on the given T and input noisy image. DDPM consists of two processes which are the Forward and Reverse/Denoise processes. Furthermore, the optimization objective is to minimize the error (typically using MSE) between the actual noise added during the forward process and the noise predicted by the model (can also be trained to directly predict the image X at t). The sampling (or generation) process then performs a progressive denoising procedure, starting from a random noise input, to generate a new image.

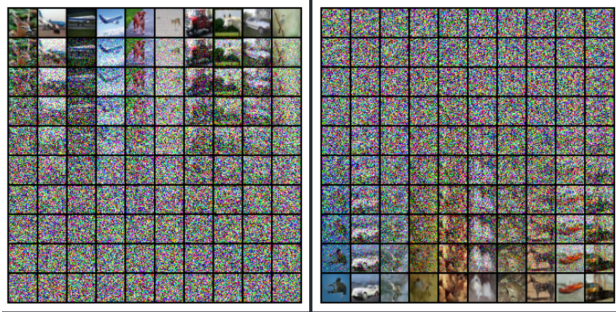


Fig. 4 Forward and Reverse process

Those two key processes have been utilized in those training and sampling algorithms. Forward process: $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$ and Reverse process: $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$, and their mathematical derivation can be explained in some principles like Markov Chain, ELBO (Evidence Lower Bound), and KL divergence.

Algorithm 1 DDPM Training

```

1: repeat
2:    $x_0 \sim q(x_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(0, I)$ 
5:   Gradient descent on:  $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, t)\|^2$ 
6: until converged

```

Algorithm 2 DDPM Sampling

```

1:  $x_T \sim \mathcal{N}(0, I)$ 
2: for  $t = T, \dots, 1$  do
3:    $z \sim \mathcal{N}(0, I)$  if  $t > 1$ , else  $z = 0$ 
4:    $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left( x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$ 
5: end for
6: return  $x_0$ 

```

A. Model Architecture

For the model design, we employ a U-Net-based[6] architecture with an encoder-decoder structure and skip connections, following the original DDPM but incorporating a smaller size (25.6M parameters trained) and simpler building blocks. The network utilizes max pooling for downsampling and nearest-neighbor interpolation for upsampling, processing features through four convolutional blocks with channels [64, 128, 256, 512]. Residual connections[7] are implemented within each block by adding the input features to the output, where 1×1 convolutions adjust the channel dimensions when necessary. Self-attention mechanisms are incorporated in the deeper layers (channels 256 to 512), where the spatial dimensions of the feature maps are reduced (from 8×8 to 4×4). The model is further enhanced with time conditioning through sinusoidal positional encoding, which transforms diffusion timesteps into 256-dimensional vectors. For conditional generation, class labels are embedded into the same 256-dimensional space and combined with the time encodings, allowing the model to generate images conditioned on specific classes.

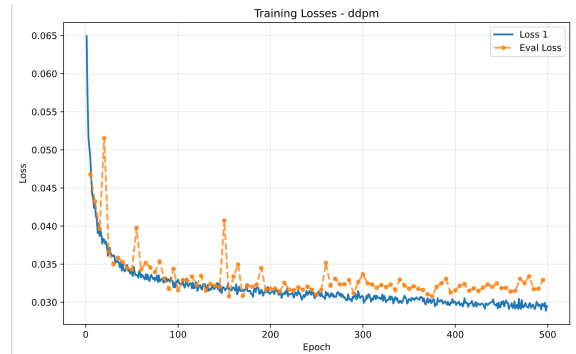
For the diffuser parameters beta and T, we just keep it same as the original parameters in DDPM, which T=1000, Beta1 = 0.0001 and BetaT = 0.02.

B. Experiment Design and Results

The model was trained on the CIFAR-10 dataset, normalized to (-1, 1) with batch size = 128, for 500 epochs (approximately 200,000 iterations). When trained on an RTX 4060 GPU, each epoch took about 60 seconds, and model parameters is saved every 50 epoches for later FID score evaluation.

Results: After 500 epochs of training, the model converged with MSE losses of approximately 0.029 on the training set and 0.032 on the test set. It is clear that after 250 epochs, the gap between training loss and test loss starts increasing. We chose to compare the FID scores of the two models saved at 250 and 500 epochs to check if evaluation loss is always an effective indicator for training image generative models. The results show that the model saved at 250 epochs achieved higher FID scores than the model at 500 epochs, which to some extent supports that MSE loss might not always be the optimal indicator for training. This further implies that the model continues to improve in the final 250 epochs, even though the evaluation loss remains at a level similar to that at epoch 250.

Metric	Epoch 250	Epoch 500
FID train	36.94	33.93
FID test	38.54	35.75

Fig. 5 Model performance at different epochs**Fig. 6** DDPM loss

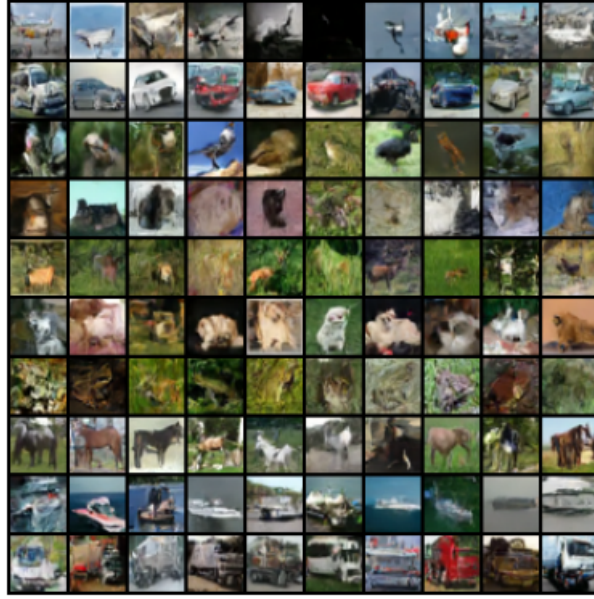


Fig. 7 conditional Cifar10 generated by DDPM 500-epoch-trained

V. Conclusion

Based on our one-month group project involving training and evaluation of the three most popular image generative models, we empirically validated the key advantages and limitations discussed in class. Our model-training results and generated outputs clearly demonstrate the following:

VAE produces relatively blurry and less sharp images, as it prioritizes stable and robust training over high-frequency detail generation.

GAN models are capable of generating high-quality, sharp images but often suffer from training instability and mode collapse.

DDPM achieves the best of both worlds: stable training dynamics and high sample quality, at the cost of a computationally expensive and slow sampling process.

Furthermore, our DDPM training results suggest that loss on test/validation datasets is not an effective evaluation metric for image generative models. In future work, dynamic metrics like the FID score should be integrated to achieve a more informed and reliable training design.

References

- [1] Kingma, D. P., and Welling, M., “Auto-Encoding Variational Bayes,” , 2022. URL <https://arxiv.org/abs/1312.6114>.
- [2] Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y., “Generative Adversarial Networks,” , 2014. URL <https://arxiv.org/abs/1406.2661>.
- [3] Ho, J., Jain, A., and Abbeel, P., “Denoising Diffusion Probabilistic Models,” , 2020. URL <https://arxiv.org/abs/2006.11239>.
- [4] Krizhevsky, A., “CIFAR-10 and CIFAR-100 datasets,” <https://www.cs.toronto.edu/~kriz/cifar.html>, 2009.
- [5] Raschka, S., Liu, Y. H., and Mirjalili, V., *Machine Learning with PyTorch and Scikit-Learn*, Packt Publishing, Birmingham, UK, 2022.
- [6] Ronneberger, O., Fischer, P., and Brox, T., “U-Net: Convolutional Networks for Biomedical Image Segmentation,” , 2015. URL <https://arxiv.org/abs/1505.04597>.
- [7] He, K., Zhang, X., Ren, S., and Sun, J., “Deep Residual Learning for Image Recognition,” , 2015. URL <https://arxiv.org/abs/1512.03385>.